AN1.4 - Tool Registry & Capability Audit (A2)

Sprint: AT24 AI Agents - Agent Framework Foundation **Step:** A2 - Tool Contract + Registry **Depends on:** AN1.1 audit, AN1.2 locked decisions, AN1.3 (A1 closure, `AF-v1`) **Gate:** G02 - must demonstrate the full Framework -> Registry -> real AT24 service -> typed result -> provenance pipe before A3.

> **Hard rule applied:** every tool below is classified from what AT24 can *actually execute today*, traced to a real callable entry point. No tool is registered to make the Agent UI look complete.

1. Method - the trace performed for every candidate

```
Tool  ->  actual AT24 service / function (file:export)
     ->  authority (who owns the execution)
     ->  input / output schema (real TS types)
     ->  permission (PermissionKey)
     ->  autonomy (floor)
     ->  evidence / provenance (what real provenance the result carries)
     ->  credits (flat estimate for v1 - placeholder value, real ledger in A9)
     ->  execution mode (sync | resumable)
     ->  classification
```

Classification vocabulary:

Class	Meaning
`READY`	A real, callable server-side entry point exists and returns a typed result with real provenance. Registered in A2.
`ADAPTER_REQUIRED`	A real capability exists but the callable surface is client-only, or is a pipeline-internal stage, or needs a thin composition. Registered only when its consuming agent is built (A11-A13), with the adapter.
`BLOCKED`	A real implementation exists but is disabled / gated off in production today.
`NOT_AVAILABLE`	Only mock data or no implementation. Not registered.

2. Capability audit

2.1 `READY` - registered in A2

Tool id	Real entry point	Input -> Output	Perm / Autonomy	Provenance the result carries	Exec
`market.snapshot`	`marketData.getSna` (`{ symbol }` -> `MarketSnapshot`) - `services/market-data/shared-instance.ts` (`MarketDataService`)	`{ symbol }` -> `MarketSnapshot`	`CAN_READ_MARKET_D` / 0	`provider`, `cached`, `cacheAgeMs`, `fallbackUsed`, `retrievedAt`, `timestamp`, `providerSymbol` - all real, router-tracked	`sync`

Tool id	Real entry point	Input -> Output	Perm / Autonomy	Provenance the result carries	Exec
market.intelligen	new RealTimeIntelligetimeframe?,).build({ userId, question, symbol, timeframe, requestId, requestedAt }) - services/intelligence/orchestration/real-time-intel	{ symbol, question? } -> VerifiedRealTimeIntelligenceContex (status, envelope, query, observability)	CAN_READ_MARKET_DATA / 0	The IntelligenceEnvelope: evidence (EvidenceBundle, same EvidenceItem vocab A1 aligned to), regime, marketState, intelligenceScore;s pipelineVersion (15D.x), intelligenceEngineVersion, cross-provider-validation summary. **Deterministic - zero LLM.** Already the production path (D2.6.5, intelligence/panel route). Persists an IntelligenceAnalysisRun (existing table).	sync
backtest.run	algoTestService.rAlgoTestRunReques { strategyId, symbol, timeframe, startTime, endTime, initialBalance }) - services/algo-test/algo-test.service.ts -> at24-quant-engine runSimulation()	-> AlgoTestRunView (metrics, trades, equityCurve, assumptions, resultHash)	CAN_RUN_BACKTEST / 0	resultHash, assumptions (spread/slippage/fee/latency = Zero models, dataFidelity), dataSource, bar counts, engine determinism. Persists an AlgoTestRun (existing table). **Narrow surface today:** strategyId="golden", symbol="XAUUSD", timeframe="5m", MAX_RANGE_DAYS=14.	sync
portfolio.read	new PaperTradingServi - services/paper-trading.service.ts	{} -> PaperAccountSummary (balance, leverage, usedMargin, positions[])	CAN_READ_PORTFOLIO / 0	Real per-user paper account state; positions carry real entryPrice/marginUsed/realizedPnl. Read-only (no openPosition/closePosition - those are L2, not v1).	sync

All four are read-only or bounded-deterministic, `autonomyFloor 0`, and prove the seams the owner asked for: **the existing Intelligence Pipeline** and **the existing Quant Engine**, called as tools - never reimplemented.

2.2 `ADAPTER_REQUIRED` - not registered in A2; built with their consuming agent

Capability	Real thing that exists	Why an adapter	Planned for
`research.knowledge_search`	Server-side: `GeminiEmbeddingProvider()` + `RepositoryFactory.vectors(userId).similar({ embedding, topK, userId, knowledgeId })` (see `app/api/private/knowledge/search/route.ts`). The `services/knowledge/KnowledgeSearchService` export is **client-only** (calls the HTTP route + a locally-loaded doc list).	Need a ~30-line server-side function: embed query -> pgvector cosine search scoped by `userId`. Thin layer (likely necessary).	**A11** (Research Agent)
`news.search`	`new AlphaVantageNewsProvider().getNewsEvidence({ symbol, asOf })` - real, `isConfigured()`-gated, returns `EvidenceItem[]`. Already feeds `market.intelligence` internally.	Thin wrapper to expose it as a standalone tool with its own schema + credit cost.	**A11 / A12**
`indicators.compute`	`calculateSeries(def, bars, params)` from `at24-quant-engine` - pure.	Needs bar-fetch (`marketData.getTimeSeries`) + `OHLCVBar` mapping + indicator-def resolution. Moderate adapter.	**A13** (Strategy Research) if needed; otherwise deferred
`market.candles`	`marketData.getTimeSeriesWithProvenance({ symbol, interval, ... })` - real, returns candles + provenance.	Generic READY, but no v1 agent consumes raw candles directly (they consume `market.intelligence`). Register when a consumer exists.	A12/A13

2.3 `BLOCKED` - real code, disabled in production

Capability	Entry point	Block
`market.microstructure`	`binanceMicrostructureProvider` / `microstructureSnapshots`	`BINANCE_MICROSTRUCTURE_DISABLED = true` in `services/microstructure/shared-instance.` (documented crash fix, D2.7.11). Do not register until re-enabled.
`risk.evaluate` (standalone)	`RiskEngineService.assess(reasoning)` - `services/ai/risk/risk-engine.service.ts`	Not standalone: it is a **pipeline stage** that requires a `ReasoningResult`. Real risk outputs already inside `market.intelligence`'s envelope (`envelope.risk`). A standalone portfolio/position-risk tool needs a new composition (own service) - that is a real build, not an adapter. Deferred to the **Risk Agent (A18/later)**; not faked in A2.

2.4 `NOT_AVAILABLE` - mock or nonexistent

Capability	Reality
`research.web_search`	**No implementation anywhere.** `grep` for web-search / serpapi / tavily / brave / bing -> zero hits. Per AN1.2 D4 it is in scope *as a replaceable adapter*, but there is no provider to wrap. **Not registered.** Built in **A11** with a real `SearchProvider` adapter + provenance, or the Research Agent ships knowledge-base + (later) news only. Owner decision point flagged in sec 5.
`strategy.library_lookup`	`services/ai/strategy.service.ts` -> `mockStrategies` (`@/data/mock-strategies`). The only real strategy artifact is `at24-quant-engine`'s single `buildGoldenStrategySpec()`. No browsable library.
`news.calendar`	`services/ai/economic-calendar.service.ts` -> mock (`getCalendar()`).
`news.getLatest` (`services/ai/news.service.ts`)	Mock (`@/data/news`). The real news path is `AlphaVantageNewsProvider` (see sec 2.2).

---

3. A2 implementation - the minimum registry surface

Built under `services/agent-framework/tools/`. Server-only. Consumes the `AF-v1` contract layer; adds the runtime seam.

3.1 A1 contract amendment (additive - does NOT bump `AF-v1`)

`types/agent-framework/tool-contract.ts` gains two fields on `ToolDefinition`, required for G02 requirements 7 and 9:

- `evidence: ToolEvidenceSpec` - `{ producesEvidence: boolean; evidenceTypes: AgentEvidenceType[]; provenanceProducer: string }`
- `executionMode: "sync" | "resumable"`

Plus `UnknownToolError`. Additive per AN1.2 ("a new optional field ... does not bump" - these are new fields on a type with zero existing instances).

3.2 The surface (maps 1:1 to the owner's 10 requirements)

#	Requirement	Where
1	Deterministic tool **registration**	`ToolRegistry.register(impl)` - throws `DuplicateToolError` on a repeat id; `freeze()` locks the registry
2	Deterministic tool **lookup**	`ToolRegistry.get(id)` / `.require(id)` / `.list()` / `.describe(id)`
3	**Input** schema validation	`ToolImplementation.parseInput(raw): ToolInputParseResult<TInput>` - hand-written guard (repo convention: no schema lib), run by `ToolGateway` before the handler. `ToolDefinition.inputSchema` (JSON Schema) stays declarative for the future Agent Builder UI / LLM specs.

#	Requirement	Where
4	**Output / result** validation	`ToolImplementation.checkOutput(value): ContractValidationResult`, run by `ToolGateway` after the handler; a failure -> `ToolResult { status: "tool_error" }`, never a fabricated pass
5	**Permission** metadata	`ToolDefinition.requiredPermissions` - surfaced by `registry.describe()`; `ToolGateway` checks it against the (A4-supplied) `PermissionPolicy` via `evaluatePermission()`
6	**Autonomy** metadata	`ToolDefinition.autonomyFloor` - `ToolGateway` checks `canRunAtAutonomy()`
7	**Evidence / provenance** metadata	`ToolDefinition.evidence` (new). Each READY tool's handler returns `{ result, evidence: AgentEvidenceDraft[] }` with a real `provenanceProducer`
8	**Credit-metering** metadata	`ToolDefinition.creditCost` (A1). `ToolGateway` computes a flat pre-estimate and stamps `ToolResult.creditsConsumed`. **Real ledger + enforcement is A9** - A2 only exposes + records the number.
9	**Sync vs resumable** metadata	`ToolDefinition.executionMode` (new). All four A2 tools are `sync`. `resumable` is declared-but-unused until A4's `tick()` runtime.
10	Deterministic **unknown-tool rejection**	`ToolRegistry.require(unknownId)` throws `UnknownToolError`; `ToolGateway.invoke({ toolId: unknown })` -> `ToolResult { status: "invalid_input", errorKind: "unknown_tool" }`

3.3 What A2 deliberately does NOT do

- No `CreditGateway` / real metering / enforcement - **A9**.
- No `MemoryGateway` - **A7**.
- No planner, supervisor, or runtime loop - **A4/A5**. `ToolGateway.invoke()` is a single authorized call; it is **not** an agent loop and takes an `AuthorizedToolIntent` (already past permission/autonomy/credit), never a raw `PlannerToolRequest`.
- No Prisma models, no migration - **A3**.
- No API routes, no `/dashboard/agents` change.
- No LLM call anywhere in the tool layer.

---

4. G02 acceptance demonstration - RESULTS

`scripts/validate-agent-tools.ts` (house style, `npm run validate:agent-tools`) demonstrates, end to end, against the **real** shared services (no mocks).

Default run: `20 passed, 0 failed`. Includes one live call:

```
LIVE market.snapshot: gateway -> MarketDataService -> typed ToolResult
     (live) XAUUSD = 4429.05127 via twelve-data
```

Full run (`RUN_LIVE_AGENT_TOOLS=1`): `22 passed, 0 failed`. All three pipeline/DB-touching tools invoked end to end through the gateway:

```
LIVE market.snapshot     -> MarketDataService        -> ok, XAUUSD 4429.05 via twelve-data, 1 evidence draft
LIVE market.intelligence -> RealTimeIntelligenceService -> ok, status "resolved", 4 real evidence drafts
LIVE portfolio.read      -> PaperTradingService       -> ok, typed PaperAccountSummary
```

The `market.intelligence` result is the flagship proof: the tool called the **existing deterministic D2.6.5 orchestrator**, got a `resolved` `VerifiedRealTimeIntelligenceContext`, and produced **4 real evidence drafts drawn only from the envelope** (regime + ranked evidence items), each with `provenance.producer = "intelligence-pipeline"` and the real `pipelineVersion`. No second intelligence implementation; no LLM.

The demonstrated pipe

```
validate-agent-tools harness
  -> buildToolRegistry()  (frozen; ids: backtest.run, market.intelligence, market.snapshot, portfolio.read)
  -> invokeTool({ intent: AuthorizedToolIntent{ toolId: "market.intelligence", input: { symbol: "XAUUSD" } }
                  permissionPolicy: { granted: ["CAN_READ_MARKET_DATA"] }, autonomyLevel: 0 })
       - registry.get           -> found
       - status === "active"    -> ok
       - evaluatePermission     -> CAN_READ_MARKET_DATA granted -> ok
       - canRunAtAutonomy(0, 0) -> ok
       - impl.parseInput(raw)   -> { ok, value: { symbol: "XAUUSD" } }
       - impl.handler(...)      -> RealTimeIntelligenceService.build(...)   [REAL, no LLM]
       - impl.checkOutput(...)  -> ok (status/query/generatedAt present)
       - validateAgentEvidenceDraft(x4) -> ok
  -> ToolResult { status: "ok", output: VerifiedRealTimeIntelligenceContext, creditsConsumed: 4 (placeholder
  -> evidence: AgentEvidenceDraft[4], provenance.producer = "intelligence-pipeline"
```

Deterministic-rejection results (no network)

Path	Result
unknown tool id	`invalid_input` / `unknown_tool`
bad input shape	`invalid_input` / `input_validation`
missing permission	`permission_denied` / `missing_permission`
autonomy floor not met	`permission_denied` / `autonomy_floor`
handler throws	`tool_error` / `handler_threw`
handler output fails `checkOutput`	`tool_error` / `output_validation`
non-`active` tool	`tool_error` / `tool_disabled`
duplicate registration	`DuplicateToolError`
register after `freeze()`	throws
malformed `ToolDefinition`	`InvalidToolDefinitionError`

G02 checklist - verified

- [x] **no fake / mock production tools** - 4 `READY` tools, each a real entry point; `ADAPTER_REQUIRED`/`BLOCKED`/`NOT_AVAILABLE` capabilities are documented, not registered
- [x] **no second Intelligence implementation** - `market.intelligence` calls the existing `RealTimeIntelligenceService`
- [x] **no second Quant / backtest engine** - `backtest.run` calls the existing `algoTestService` -> `at24-quant-engine`

- [x] **no provider-specific contract leakage** - `types/agent-framework/*` still imports only relative `./` paths (asserted); the A1 amendment added `evidence`/`executionMode` types, no vendor names. Vendor SDK use (`GeminiEmbeddingProvider` etc.) exists only in *future adapter implementations*, never a contract type
- [x] **no planner direct execution** - `invokeTool()` takes an `AuthorizedToolIntent`; the planner types (`PlannerToolRequest`) stay inert; the gateway re-checks permission + autonomy as defence-in-depth and never re-plans
- [x] **no live execution** - no `EXECUTION`-category tool registered; `portfolio.read` is read-only (`getSummary` only, no `openPosition`/`closePosition`)
- [x] **no credit bypass** - every `invokeTool()` path stamps `ToolResult.creditsConsumed` from `ToolDefinition.creditCost`; the number is a flagged placeholder, the *seam* is exercised. Real ledger + enforcement is A9
- [x] **no mutation of legacy `/dashboard/agents`** - not touched
- [x] **no A3 Prisma work** - no schema change, no migration; `backtest.run` / `market.intelligence` trigger only their wrapped service's *own* existing persistence
- [x] **tsc** - `npx tsc --noEmit` reports 0 errors in `types/agent-framework/**`, `services/agent-framework/**`, `scripts/validate-agent-*.ts`. (77 repo-wide errors, all in the stale generated `.next/dev/types/validator.ts`, unrelated.)
- [x] **A1 contract tests still green** - `npm run validate:agent-contracts` -> `38 passed, 0 failed` (fixture updated for the additive `evidence`/`executionMode` fields)

---

5. Owner decision points (before A11, not before A3)

1. **`research.web_search`** - build a real `SearchProvider` adapter in A11 (which vendor? cost? provenance = result URLs), or ship the Research Agent on knowledge-base + `news.search` only for v1?
2. **`backtest.run` surface** - keep the golden/XAUUSD/5m surface for the Strategy Research Agent's first end-to-end proof (A13), widen later? (Widening is additive in `algoTestService`.)
3. **`risk.evaluate`** - confirm the standalone Risk Agent tool is out of scope until the Risk Agent step, and v1 agents rely on `market.intelligence`'s `envelope.risk` for risk context.

---

6. Files changed (A2 implementation - delivered)

Added (10):

```
frontend/services/agent-framework/tools/tool-implementation.ts     ToolImplementation type + ToolHandlerCont
frontend/services/agent-framework/tools/tool-registry.ts           ToolRegistry (register/get/require/list/d
frontend/services/agent-framework/tools/tool-gateway.ts            invokeTool() - the single authorized-call
frontend/services/agent-framework/tools/tool-credit-costs.ts       PLACEHOLDER flat costs, flagged for the A
frontend/services/agent-framework/tools/impl/market-snapshot.tool.ts
frontend/services/agent-framework/tools/impl/market-intelligence.tool.ts
frontend/services/agent-framework/tools/impl/backtest-run.tool.ts
frontend/services/agent-framework/tools/impl/portfolio-read.tool.ts
frontend/services/agent-framework/tools/registry-manifest.ts       buildToolRegistry() + frozen `toolRegistr
frontend/scripts/validate-agent-tools.ts                           G02 validation (20 default / 22 with RUN_
```

Modified (3):

```
frontend/types/agent-framework/tool-contract.ts    + ToolEvidenceSpec, ToolExecutionMode, UnknownToolError;
frontend/types/agent-framework/evidence-contract.ts + AgentEvidenceDraft + validateAgentEvidenceDraft  (ADDI
frontend/scripts/validate-agent-contracts.ts        fixture + 4 cases for the new tool fields
frontend/package.json                               + "validate:agent-tools"
```

Not touched: Prisma schema, every existing route/service/component, the legacy `services/agents/*` + `types/agent.ts` + `/dashboard/agents`.

6.1 A1 contract amendment - rationale

`AF-v1` is unchanged: per AN1.2 ("a new optional field ... does not bump" the contract version), the two new `ToolDefinition` fields and the `AgentEvidenceDraft` type are additive on types with **zero pre-existing instances**. They were required by the owner's own G02 registry-surface list (requirement 7 - evidence/provenance metadata; requirement 9 - sync/resumable metadata). No existing behavior changes.

7. Test commands

```
npm run validate:agent-contracts     # A1 - 38 passed, 0 failed
npm run validate:agent-tools         # A2 - 20 passed, 0 failed (1 live: market.snapshot)
RUN_LIVE_AGENT_TOOLS=1 npm run validate:agent-tools   # A2 full - 22 passed, 0 failed (3 live)
```

8. Status

- Capability audit complete (sec 2). 4 `READY`, 4 `ADAPTER_REQUIRED`, 2 `BLOCKED`, 4 `NOT_AVAILABLE`.
- A2 registry surface + gateway + 4 real tools **implemented and committed** (sec 6).
- G02 pipe **demonstrated end to end against real services** (sec 4): `market.intelligence` returned a `resolved` context with 4 real evidence drafts; `market.snapshot` returned a live XAUUSD quote; all deterministic-rejection paths verified.
- 3 owner decision points (sec 5) are gated on **A11-A13**, not A3.

Awaiting G02 review. On acceptance -> A3 (Run persistence + Prisma migration, generated NOT applied).

End AN1.4.